

Hands on ML – PyTorch Version Journal

Chapter 1

Machine Learning: Computer learning from data

Why do we need Machine Learning?

Traditional programming would be way too time consuming in writing down or hard coding in every case that helps label or classify the problem. As a result of hard coding in every case, if a new case came in, the program couldn't be able to classify it. while in machine learning the model will learn to catch new patterns

Example:

If I'm writing a program to classify spam emails and i wrote that if the email contains 4U it's a spam, but the email I received include "for U" the program wouldn't be able to classify it

Machine learning digs into large amount of data to discover hidden patterns --

> Data mining

So instead of writing each word that is a suspect of being in a spam email , machine learning digs into large amounts of data , discovers the patterns and can be later inspected to reveal the list of words or combinations of words that makes up a spam email

Classifying Machine Learning based on:

- Guidance during training (Supervised, unsupervised.)
- How they learn (Batch/online)
- Do they detect patterns and build a model, or compare new points to data points? (instance-based or model-based)

Guidance during training

1-Supervised Learning:

Training the algorithm with the desired solution (labels for classification, target for regression)

Classification: predicts a label (categorical data)

Regression: predicts a target (numerical data)

Some regression can be used for classification and the other way around, like logistic regression, it is used for classification, but it outputs numerical values which shows the probability of being in a certain class

2-Unsupervised Learning:

Training the algorithm with unlabeled data

Examples:

Dimensionality Reduction --> simplifying the data without losing information can be done by feature extraction (merging of more than 1 correlated feature into one) or feature selection (neglecting some of the features)

Visualization Algorithms --> Type of Dimensionality Reduction that uses feature extraction to reduce the dimensions to visualize the data in 2D or 3D representation

Why do we reduce the dimensions or features of our data?

As it will run much faster and perform much better, avoids overfitting too

Anomaly detection --> detecting the odd data, that deviates from the normal pattern. Can be used to detect outliers and remove them before training an algorithm

Novelty Detection --> detects instances that looks different from all instances in the training set

Difference between anomaly and novelty?

If I have a dataset of dogs and 1% of the dogs are of bulldog breed

Anomaly detection would classify them as anomalies as they are of very small amount, while novelty wouldn't as it compares it to all instances if at least 1 is there so it's no different than the data I have.

Association Rule --> Digs into the data to discover relations between features
example: if someone like fries then he'll probably like ketchup too

So, you'll be placing them next to each other in a supermarket

Clustering Algorithms and Generative are also types of unsupervised learning

3-Semi-supervised Learning:

Trains partially labelled data

Combines supervised and unsupervised algorithms, so starts by clustering similar instances into groups then every unlabeled instance can be labeled with the most common label in its cluster. then you can perform any supervised learning algorithm on it

Example is the photo album one, it collects similar faces into groups and asks you to label them to if it receives another new instance, it will know where to put it

Can start with supervised then move to unsupervised but its not common

4-Self-supervised Learning:

Labelling a fully unlabeled dataset

Example: if you have a mask or something hiding a picture, the model will be able to recover it to the original image.

This is usually part of the model not the whole one , for instance if you want to classify the breed of a pet so you'll build a model to remove any unwanted things from the picture then put it on another model to predict its breed -->

Transfer Learning : Reusing the knowledge of a task to a new task

Different from xgboost and adaboost as these improve the performance of a task using ensembles

4-Reinforcement Learning:

Observes the environment, performs an action, learns by receiving reward or penalties

How do they learn from an incoming stream of data?

1-Batch Learning (offline learning):

Trained from a whole dataset, taking a lot of time, then it's launched into production and runs without learning anymore

Disadvantages:

Takes too much time

Model performance can decay as data is changing over time (data drift)

How fast can it start to decay? Based on your problem if it depends on data that's usually changing or not

2-Online Learning:

System is being trained incrementally by constantly giving it individual data or by small groups (mini batches), so it keeps on learning during production

Advantages:

Minimal computing resources

Adapts to rapidly changing data

Parameter --> Learning Rate

High learning rate will make the model quickly forget the old data -->

Catastrophic Forgetting

Low learning rate will learn very slowly and be less sensitive to outliers

Disadvantages:

Performance would decline if it received bad data

Out-of-core learning algorithms train datasets that do not fit in main memory by processing the data in small mini-batches and updating the model incrementally using online learning techniques.

Out-of-core learning is **not inherently online in production**. It's mostly a **training technique** for big data. Online learning, on the other hand, is explicitly designed for **incremental updates in production**.

Out of core is not a type of online

Do they detect patterns and build a model, or compare new points to data points? How they generalize

1-Instance-Based:

Model learns by heart, then it generalizes to new cases by using a similarity measure to compare new examples with the old ones

Best used with:

Small datasets, as it will require always having all the data for every prediction since it does that, it is very slow and not suitable for high-dimensional data such as images

2-Model-Based:

Generalizes by building a model, then using it to make predictions.

So, we build a model, get our parameters, improve their performance either by utility measure (how good it is) or by cost function (how bad it is)

Main Challenges in Machine Learning

We're training a model and, on some data, so the 2 things that can go wrong are the model and data.

Data

1-Non-representative data

Data should be representative of the cases you want to generalize, it's possible to have some sampling noise but if very large amount of data is non-representative due to flawed sampling method --> sampling bias

2-Poor data quality

Data is full of errors, outliers, and noise --> making it harder for the system to detect the pattern

Difference between outliers and noise:

Outliers is a very large or small number that is so much different than the rest of the data and might as well affect the model

Noise is a slight change in the pattern that wouldn't affect the model as much
example: having a house price model $2x+10$

But 1 sample is 110 is 900\$, which doesn't match the data

While a noise could be 140 is for 180

3-Irrelevant Features

Model will learn best when we have relevant features and not too many irrelevant ones

Coming up with good features to train on --> **Feature Engineering**

Feature Selection, Feature Extraction, create new features by gathering new data

4-Insufficient quantity of training data

Get a suitable amount of data, depending on your problem

Model Challenges:

1-Overfitting

Model performs well on the training data, but doesn't generalize well

Fixing overfitting:

Simplify the model (go from high-degree polynomial to a linear regression)

Reduce the number of attributes / features

Constraining the model → limiting the freedom(θ) of the model so that it doesn't fit every noise or fluctuation

Common methods --> Regularization Techniques

Gather more training data

Reduce the noise in the training data

In case of having small data, this could cause overfitting or underfitting:

Overfitting if the model is complex and data is noisy

Underfitting if the model is too simple and pattern is complex

2-Underfitting

Model is too simple to capture the pattern

Fixing it:

Choose more powerful model

Get better features

Reduce constraints on the model (as regularization works on making the model simple, which is the opposite of what we want)

Another type of issues may arise --> **Deployment Issues**

Issues like:

Model is too complex, has a lot of hyperparameters, ensemble of many complex models as a result making it harder to maintain

Model is too large to fit in the memory

Model is too slow as a result response time is so high

Model can't scale properly, can handle only few users

Model could have security vulnerabilities

Model could become outdated

There comes the role for MLOps Team

Testing and Validation

Test how well the model generalized to new data

Training and test set

Error rate on new cases is called generalization error or out of sample error

Usually divide them into 80 , 20

As the size of the dataset increases, the test set can represent a smaller percentage of the data, since a small fraction of a large dataset is still sufficient to reliably estimate the generalization error.

Problem:

If you want to choose the best regularization parameter, so you trained 100 different models and you tested them till you found one with 5% error and you picked it

Now when this model went to production, the model produced error rate of 15%

Why did this happen? As the model learned on both training and testing set so it is no longer new data it hasn't seen

Solution:

Holdout validation:

Chooses model

Chooses hyperparameter

If the validation is too small, then the model is imprecise

If the training set is too small, the model cannot learn the underlying patterns properly, while if the validation or test set is too small, the model's performance estimate becomes unreliable because it is evaluated on too few unseen examples.

Another problem:

If we have trained a model and we've got high validation error, now we don't know the problem is because we have data mismatch between training and validation or the model overfit the data

Solution:

We split the training data into training set and train-dev set and test on train-dev

If the train-dev produced high error --> model overfit

If the train-dev produced low error --> data mismatch

Stop when you have a model that performs well on both train-dev and validation

Chapter 3

Classification

Evaluating Classification Models:

1-Accuracy:

If you have unbalanced dataset and you're testing if that label belongs to a certain class, if that label is only 10% of the data and every guess turns out to be the other class so this gives a 90% accuracy, but that doesn't mean that that model is performing well

So, this performance measure is not preferred performance, especially when dealing with skewed data

So, we move to

2-Confusion Matrix

Shows the number of true positive, true negatives (these are the main diagonal and is best when it is a non-zero)

Also shows false positive, false negatives (incorrectly classified)

The second part is the prediction, and the first part is is it correctly classified or not

Ex: True positive (the model predicted it as positive, and it is true positive)